

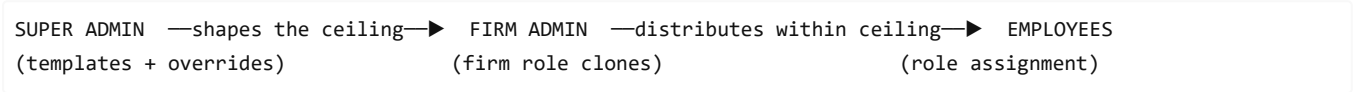
Delegation Chain — Operator & Frontend Guide

Super Admin → Firm Admin → Employees (Law Firm ERP)

Version 1.0 — 2026-09-08 — branch `devG` Companion spec: `docs/rbac-delegation-chain-design.md` · Verified by: `docs/e2e-run-log.md` (6/6 E2E steps passed)

1. What this feature is

The platform enforces a **three-level permission chain**:



The invariant, enforced at every point in time: no employee ever holds a permission their Firm Admin doesn't hold; no Firm Admin ever holds a permission the Super Admin hasn't allowed via the `FIRM_ADMIN` system template.

Core decisions behind the design (asked & answered during spec):

- Role-level only** — no per-user permission grants. "John needs edit access" → Firm Admin creates a custom role (e.g. *Senior Paralegal*) and reassigns John. Two or more people need it → change the role.
- Diff-based sync** — SA template edits propagate only the delta (added/removed permissions). Firm customizations SA never touched survive.
- Async fan-out** — template edits return immediately with a `syncJobId` ; propagation happens in the background so request latency never scales with customer count.

2. What changed & what's new (change log)

Changed behavior

Area	Change
System role templates	Were immutable — now SA-editable via the template endpoints (all except <code>SUPER_ADMIN</code> , which stays permanently immutable).
Boot seeding	Templates edited by SA are frozen (<code>last_sa_edit_at</code> set): the seeder never re-injects default matrix values into them. Before this, a restart silently resurrected removed permissions.
Custom role creation	Now accepts/validates <code>parentRoleId</code> (base system template; not <code>SUPER_ADMIN</code> ; must be active). Legacy roles get backfilled at boot only when their lineage is certain.
Firm Admin self-narrowing	When a Firm Admin removes permissions from their own <code>FIRM_ADMIN</code> role, employees holding those permissions are automatically stripped — and the response body says exactly what was removed (<code>cascadeEffect</code>).
PUT <code>/admin/roles/permissions</code>	Parent-template ceiling now governs template chain validation only; SA's per-firm override endpoint stays ceiling-free.

New APIs (all SA-guarded unless noted)

Method	Endpoint	Purpose
GET	<code>/api/v1/admin/roles/templates</code>	List all system role templates with current permissions
GET	<code>/api/v1/admin/roles/templates/{id}/permissions</code>	One template's permissions + <code>lastSaEditAt</code>
GET	<code>/api/v1/admin/roles/templates/{id}/permissions/preview</code>	Dry-run a template edit (zero writes)

PUT	/api/v1/admin/roles/templates/{id}/permissions	Edit a template → returns syncJobId
GET	/api/v1/admin/roles/sync-jobs/{jobId}	Poll sync job status
GET	/api/v1/super-admin/firms/{firmId}/roles	SA view of a firm's roles + permissions + user counts
POST	/api/v1/super-admin/firms/{firmId}/roles	SA creates a custom role inside a firm

New DTO fields the frontend should render

Field	Where	Meaning
syncJobId	PUT template response	Poll this job before refreshing permission views
lastSaEditAt	template reads	Template is SA-customized (seed matrix no longer applies)
removedPermissionCodes, addedPermissionCodes	preview	The exact delta
firmImpacts[].cloneImpacts[].skippedByCeiling	preview	Adds that will be skipped for that firm (its admin lacks them)
firmImpacts[].cloneImpacts[].cascadeStripped	preview	Permissions the cascade will remove from employees
employeeTemplateStrips	preview	Over-ceiling permissions stripped from employee templates platform-wide
chainValidationViolations / wouldViolateChain	preview	Show these before allowing commit
cascadeEffect	firm-role PUT response	What the self-narrowing cascade removed, per role

New data

- Table `sync_jobs` (status: `PENDING` → `RUNNING` → `COMPLETED` | `COMPLETED_WITH_FAILURES` | `FAILED`).
- Column `roles.last_sa_edit_at` .

3. The API walkthrough (tested step by step)

Every step below was executed against the running app — see the E2E log in §7. Base URL: `{baseUrl}` (dev: `http://localhost:6969`). All bodies are wrapped: `{ "data": { ... } }` . Auth: `Authorization: Bearer <accessToken>` .

STEP 0 — Onboarding (existing APIs, unchanged)

0.1 SA creates the firm + first admin

```
POST /api/v1/super-admin/firms
{ "data": {
  "lawFirmCode": "FIRMA", "name": "Firm A", "firmType": "FIRM",
  "adminUsername": "adminA", "adminEmail": "adminA@test.com",
  "adminMobileNo": "9820000002", "adminPassword": "AdminPass123!",
  "adminFullName": "Admin A" } }
```

→ `data.firmId` . Role clones for FIRM_ADMIN/ADVOCATE/PARALEGAL/CLIENT are created **empty** (0 permissions).

0.2 Firm Admin logs in → `POST /api/v1/auth/login` (firm login) → `accessToken` . **0.3 Employees** are created by the Firm Admin (existing employee endpoints) and hold a role clone (e.g. PARALEGAL) — also 0 permissions until Step 3.

STEP 1 — SA populates the Firm Admin (*existing override API*)

```
PUT /api/v1/super-admin/firms/{firmId}/roles/{firmAdminRoleId}/permissions
{ "data": { "roleId": "{firmAdminRoleId}",
            "permissionIds": ["<uuid CASE_MANAGEMENT:VIEW>", "<uuid CASE_MANAGEMENT:CREATE>",
                              "<uuid BILLING:VIEW>", "<uuid USER_MANAGEMENT:EDIT>", "..."] } }
```

- 200 with the role's new permission list. **Note:** `roleId` is required in the body on this endpoint (existing contract).
- ⚠ This bumps the admin's `permissionVersion` → **the admin's current token becomes invalid (401 on next call) — re-login required.** Same for anyone holding the edited role.

STEP 2 — SA sees the firm's roles (*new*)

```
GET /api/v1/super-admin/firms/{firmId}/roles
```

→ `data: [ { id, name, code, isActive, userCount, permissions: [ {code, action, scope} ] } ]` Use this to discover role ids before overrides, and to audit what a firm has done.

STEP 3 — Firm Admin distributes within their ceiling (*existing API, unchanged*)

3.1 Read current + ceiling (drives the checkbox UI):

```
GET /api/v1/firm/roles/{roleId}/permissions
```

→ `data.currentPermissions` + `data.availablePermissions` (each with `assigned: true|false`).

3.2 Save the employee role's permissions:

```
PUT /api/v1/firm/roles/{roleId}/permissions
{ "data": { "roleId": "{roleId}", "permissionIds": ["<uuid>", "..."] } }
```

- Server rejects anything above the Firm Admin's own set (403) — the ceiling cannot be exceeded.
- If the edited role **is** the Firm Admin's own and permissions were removed, the response includes `cascadeEffect: [ "role PARALEGAL: removed [CASE_MANAGEMENT:CREATE]", ... ]` — **render this to the admin.**

STEP 4 — The John/Ron case (custom role) (*existing API + new anchor field*)

```
POST /api/v1/firm/roles
{ "data": { "name": "Senior Paralegal", "code": "SENIOR_PARALEGAL",
            "parentRoleId": "<PARALEGAL system template id – optional but recommended>" } }
```

- `parentRoleId` must be a system template (not SUPER_ADMIN, active). Wrong anchor → 400/404.
- Then assign the employee: existing bulk-role-change endpoint. GET the template id from Step 5.1.

STEP 5 — SA reshapes the platform (templates) (*new*)

5.1 List templates:

```
GET /api/v1/admin/roles/templates
```

→ all five templates with permissions. Use `id` for the calls below.

5.2 Preview (ALWAYS call before PUT — zero writes):

```
GET /api/v1/admin/roles/templates/{templateId}/permissions/preview
{ "data": { "permissionIds": ["<full new set for this template>"] } }
```

→ delta (addedPermissionCodes / removedPermissionCodes), per-firm firmImpacts (added / removed / skippedByCeiling / cascadeStripped), employeeTemplateStrips , and wouldViolateChain + chainValidationViolations . **UI rule: if wouldViolateChain is true, block commit and display the violations.**

5.3 Commit:

```
PUT /api/v1/admin/roles/templates/{templateId}/permissions
{ "data": { "permissionIds": ["<full new set>"] } }
```

→ 200 immediately with syncJobId . **The propagation is async.**

5.4 Poll:

```
GET /api/v1/admin/roles/sync-jobs/{syncJobId}
```

→ status , firmsTotal , firmsCompleted , firmsFailed , errorSummary . Poll ~2s intervals until COMPLETED / COMPLETED_WITH_FAILURES / FAILED , then refresh any cached permission data. COMPLETED_WITH_FAILURES lists failed firm ids in errorSummary .

Rules the backend enforces (frontend should mirror them in the UI):

- SUPER_ADMIN template → PUT rejected (immutable by design).
- Employee template set must stay **within** the FIRM_ADMIN template set → violation = 400 naming the offending codes.
- FIRM_ADMIN template cannot be narrowed **below** what an employee template holds → 400 telling you to narrow the employee template first. (E2E verified: removing BILLING:VIEW from FIRM_ADMIN correctly fails because the ADVOCATE template holds it.)
- GLOBAL -scope permissions are rejected everywhere here.

STEP 6 — SA creates a role on a firm's behalf (new)

```
POST /api/v1/super-admin/firms/{firmId}/roles
{ "data": { "name": "Litigation Support", "code": "LIT_SUPPORT",
            "parentRoleId": "<base template id – required on this endpoint>" } }
```

Then assign permissions via the Step 1 override endpoint.

4. Error handling quick reference

Status	Meaning	Frontend action
400 Edit violates the delegation chain: ...	Chain validation failed; message names template(s) + codes	Show message; suggest narrowing employee template first
400 permissionIds list is required / invalid id	Body validation	Fix payload
401 permissions have changed	Token's permissionVersion is stale	Re-login (redirect to login)
403 exceeds the ceiling...	Firm Admin tried to grant beyond their own set	Trim selection to availablePermissions
403 SUPER_ADMIN template is immutable	Attempted SA-template edit	Hide/disable that row in UI
404	Firm/role/job not found	Refresh lists

5. Suggested UI screens (minimum viable)

1. **SA → Template Management:** table of templates (Step 5.1) → edit drawer = checkbox tree from `GET /admin/permissions/grouped` , filtered to the template's ceiling; always show Preview results before Save; after Save show a job-status banner fed by 5.4 polling.
2. **SA → Firm detail:** firm roles table (Step 2) + per-role override editor (Step 1) + "create role for firm" (Step 6).
3. **Firm Admin → Roles & Permissions:** existing screen + render `cascadeEffect` returned by saves + custom-role dialog with optional base-template picker (Step 4).

6. E2E verification (what was tested)

Automated suite `DelegationChainE2ETest` — real HTTP through the security filter chain, real async job execution: **6/6 steps passed**; full backend suite **266/266 green** (2026-09-08, branch `devG`).

Step	Verified
0	Firm onboarding; both role clones start at 0 permissions
1	SA override grants the FIRM_ADMIN clone 10 permissions
2	SA sees the firm's roles with permissions + holder counts
3	Firm Admin grants PARALEGAL a 4-permission subset within ceiling
4	Custom role with template anchor created; bad anchor rejected 404
5	Template edit: preview → commit → async job COMPLETED; cascade narrowed firm admin clone; employee clone stable; template stripped
6	Chain validation both directions: legal add 200; illegal narrowing 400 naming <code>CALENDAR:VIEW</code>

Full log: `docs/e2e-run-log.md` . Notable catch during testing: narrowing FIRM_ADMIN by `BILLING:VIEW` was **correctly rejected** (ADVOCATE template holds it) — the invariant firing in production code.

7. Out of scope / deliberately not built

- Per-user permission grants (role-level only — see §1).
- Firm-invented permission codes (only recombination of existing codes into custom roles).
- Deny/negative permissions; "respect local edits" sync mode (diff-sync chosen instead).
- Legacy `UserRole` table cleanup.